

# Manual de Usuario — TaskScript

## 1. Requisitos

| Componente        | Versión mínima                 | Notas                           |
|-------------------|--------------------------------|---------------------------------|
| Compilador C++    | g++ 9 / clang++ 12 / MSVC 2019 | C++17 obligatorio.              |
| Qt                | 6.x (Widgets)                  | También compatible con Qt 5.15. |
| CMake             | 3.16                           | Recomendado 3.21+               |
| Sistema operativo | macOS / Linux / Windows        |                                 |

En macOS se recomienda instalar Qt vía Homebrew ( `brew install qt` ). En Linux con Ubuntu/Debian: `sudo apt install qt6-base-dev cmake build-essential`. En Windows se sugiere instalar Qt desde el instalador oficial y usar el comando `cmake --build` desde el "Qt Command Prompt".

## 2. Compilación paso a paso

```
# 1) Clonar el repositorio
git clone https://github.com/<usuario>/LFP_P2_1S2026_202403125.git
cd LFP_P2_1S2026_202403125

# 2) Configurar la compilación con CMake
cmake -S Proyecto2/src -B build -DCMAKE_BUILD_TYPE=Release

# 3) Compilar
cmake --build build -j

# 4) Ejecutar
./build/TaskScriptApp          # Linux / Windows
open build/TaskScriptApp.app   # macOS
```

Si CMake no encuentra Qt, agregar la pista a `CMAKE_PREFIX_PATH`:

```
cmake -S Proyecto2/src -B build -DCMAKE_PREFIX_PATH="$(brew --prefix qt)".
```

## 3. Estructura de un archivo `.task`

Un archivo válido sigue esta estructura:

```
TABLERO "Nombre del Proyecto" {
  COLUMNA "Nombre de la columna" {
    tarea: "Nombre de la tarea" [
      prioridad: ALTA | MEDIA | BAJA,
      responsable: "Nombre del responsable",
      fecha_limite: AAAA-MM-DD
    ]
  };
};
```

Reglas importantes:

- El bloque raíz **siempre** es `TABLERO "Nombre" { ... };` (incluyendo el `;`).
- Cada `COLUMNA` se cierra con `};`.
- Las tareas dentro de una columna se separan con `,` y se permite una coma colgante antes del `}` (para coincidir con el ejemplo del enunciado).
- Las prioridades válidas son `ALTA`, `MEDIA` y `BAJA` (mayúsculas).
- Las fechas usan formato `AAAA-MM-DD` con mes 01-12 y día 01-31.

## Ejemplo 1 — Proyecto LFP

Archivo: [Proyecto2/examples/proyecto\\_lfp.task](#)

```
TABLERO "Proyecto LFP" {
  COLUMNA "Por Hacer" {
    tarea: "Disenar AFD" [prioridad: ALTA, responsable: "Jorge", fecha_limite:
2026-05-01],
    tarea: "Implementar Lexer" [prioridad: ALTA, responsable: "Maria",
fecha_limite: 2026-05-08],
    tarea: "Escribir casos de prueba" [prioridad: MEDIA, responsable:
"Carlos", fecha_limite: 2026-05-10],
  };
  COLUMNA "En Progreso" {
    tarea: "Disenar GUI" [prioridad: MEDIA, responsable: "Ana", fecha_limite:
2026-05-05],
  };
  COLUMNA "Completado" {
    tarea: "Investigar Qt" [prioridad: BAJA, responsable: "Pedro",
fecha_limite: 2026-04-20],
    tarea: "Configurar GitHub" [prioridad: BAJA, responsable: "Jorge",
fecha_limite: 2026-04-18],
  };
};
```

```
};  
};
```

## Ejemplo 2 — Sprint Planning

Archivo: [Proyecto2/examples/sprint\\_planning.task](#)

Cuatro columnas ( Backlog , Sprint Actual , Revision , Hecho ) con ocho tareas distribuidas entre cinco responsables. Excelente para validar la barra de carga del Reporte 2.

## 4. Uso de la aplicación

### 4.1 Pantalla principal

La ventana se compone de:

1. **Barra de herramientas superior** con los botones de acción:  
Cargar archivo , Guardar como , Analizar , Generar reportes HTML , Generar árbol DOT y Abrir carpeta de salida .
2. **Editor de código** (mitad superior) — admite escribir o pegar el contenido `.task` .
3. **Pestañas inferiores**:
  - **Tokens**: tabla con No. , Lexema , Tipo , Línea , Columna .
  - **Errores**: tabla con No. , Lexema , Tipo , Descripción , Línea , Columna , Gravedad .
4. **Barra de estado** que reporta el último mensaje (verde para éxito, rojo para error).

### 4.2 Flujo recomendado

1. **Cargar archivo**: pulsar el botón homónimo y elegir un `.task` . El editor se llena con el contenido.
2. **Analizar**: pulsar `Analizar` . La aplicación tokeniza, ejecuta el parser y llena ambas pestañas.
3. **Si hay errores**: revisar la pestaña *Errores*. Cada error indica línea, columna y descripción exacta. Corregir el archivo en el editor y volver a pulsar `Analizar` .
4. **Si no hay errores**: el botón `Generar reportes HTML` se habilita. Pulsarlo abre los dos reportes en el navegador predeterminado.
5. **Generar árbol DOT**: produce `arbol.dot` en la carpeta de salida. Para convertirlo a imagen ejecutar:

```
dot -Tpng arbol.dot -o arbol.png
```

## 4.3 Carpeta de salida

Todos los artefactos se guardan en `salida_taskscript/`, ubicada junto al ejecutable:

| Archivo                       | Contenido                          |
|-------------------------------|------------------------------------|
| Reporte_Kanban.html           | Reporte 1 — Tablero Kanban Visual. |
| Reporte_CargaResponsable.html | Reporte 2 — Carga por Responsable. |
| arbol.dot                     | Árbol de derivación en Graphviz.   |

Use el botón `Abrir carpeta de salida` para abrir esa carpeta en el explorador del sistema.

## 5. Interpretación de los reportes

### 5.1 Reporte 1 — Tablero Kanban

- Cada columna del archivo `.task` se representa como una columna del tablero, dispuesta horizontalmente.
- Las tarjetas muestran: nombre de la tarea, badge de prioridad (rojo/amarillo/verde para `ALTA / MEDIA / BAJA`), responsable y fecha límite.
- Una columna sin tareas se muestra con la nota *"Sin tareas en esta columna"*.

### 5.2 Reporte 2 — Carga por Responsable

- Cada fila representa a un responsable extraído de las tareas.
- `Total` es la cantidad de tareas asignadas.
- `ALTA / MEDIA / BAJA` son contadores con códigos de color.
- `Distribución` es una barra cuyo ancho corresponde al porcentaje de tareas del responsable sobre el total del tablero.
- Las filas se ordenan por carga descendente, lo que permite identificar rápidamente sobrecargas.

### 5.3 Árbol de derivación

arbol.dot puede abrirse con cualquier visor Graphviz o convertirse a imagen:

```
brew install graphviz # macOS
sudo apt install graphviz # Ubuntu / Debian

dot -Tpng salida_taskscript/arbol.dot -o salida_taskscript/arbol.png
dot -Tsvg salida_taskscript/arbol.dot -o salida_taskscript/arbol.svg
```

Los nodos no terminales aparecen en azul oscuro ( #2E75B6 ) con texto blanco; los terminales en azul claro ( #D6EAF8 ) con texto oscuro, siguiendo el ejemplo del enunciado.

## 6. Tabla de errores

Si el archivo contiene problemas, la pestaña *Errores* mostrará algo como:

| No. | Lexema               | Tipo       | Descripción                                                 | Línea | Col | Gravedad |
|-----|----------------------|------------|-------------------------------------------------------------|-------|-----|----------|
| 1   | \$                   | Léxico     | Caracter no reconocido '\$'.                                | 5     | 12  | ERROR    |
| 2   | "Tarea<br>sin cerrar | Léxico     | Cadena no cerrada antes de fin de línea/archivo.            | 8     | 20  | CRITICO  |
| 3   | ALTA                 | Sintáctico | Se esperaba ':' después de 'prioridad', se encontro 'ALTA'. | 12    | 30  | ERROR    |

El analizador **nunca se detiene** al primer error; reporta todos en una sola pasada para que el usuario corrija el archivo de forma eficiente.

## 7. Solución de problemas

| Problema                           | Solución                                                                                                  |
|------------------------------------|-----------------------------------------------------------------------------------------------------------|
| cmake no encuentra Qt6             | Definir CMAKE_PREFIX_PATH apuntando a la instalación de Qt.                                               |
| dot: command not found             | Instalar Graphviz ( brew install graphviz / apt install graphviz ).                                       |
| Reportes no se abren               | Verificar que el navegador predeterminado del SO esté configurado. La GUI usa QDesktopServices::openUrl . |
| GUI muestra texto borroso en HiDPI | Iniciar con ./TaskScriptApp -platform cocoa (mac) o ajustar QT_AUTO_SCREEN_SCALE_FACTOR=1 .               |

## 8. Atajos rápidos

| Acción           | Atajo                                      |
|------------------|--------------------------------------------|
| Cargar archivo   | Archivo → Cargar archivo (toolbar).        |
| Guardar como     | Archivo → Guardar como .                   |
| Analizar         | Análisis → Analizar o botón en la toolbar. |
| Generar reportes | Análisis → Generar reportes HTML .         |
| Generar árbol    | Análisis → Generar árbol DOT .             |

## 9. Casos de prueba documentados

Los 10 casos de prueba (válidos, errores léxicos, errores sintácticos, casos múltiples y casos borde) se documentan en [Proyecto2/tests/README.md](#) con entrada, salida esperada y resultado obtenido.